

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Services and Systemd

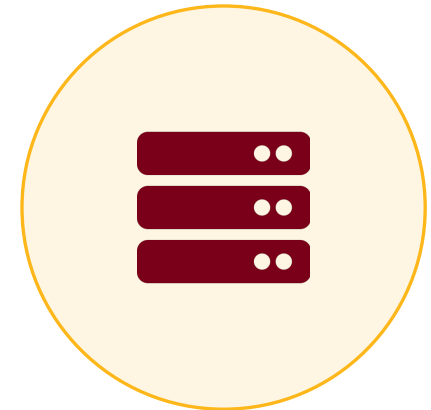
Unit 9 • Linux (10-151-105)



Learning Outcomes

By the end of this unit, you will be able to:

- Explain how systemd initializes a Linux system and manages its resources as units.
- Start, stop, enable, disable, and mask services with systemctl.
- Read a service unit file and write a simple custom service of your own.
- Choose and set the system's boot target.
- Investigate service behavior and read logs with journalctl.



LESSON 1

systemd Fundamentals



What Is systemd?

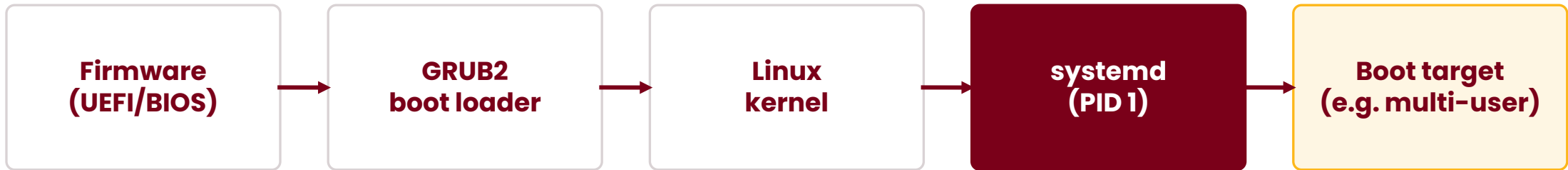
- **The init system — the first program the kernel starts and the last to stop.**
- Runs as PID 1 and is, in effect, the parent of every other process. Nothing else runs without it.
- Brings up the rest of the system at boot, then stays running as the manager you talk to.
- Standard on modern distributions, including Rocky Linux 10 and Ubuntu Server 24.04.
- Written all lowercase — systemd, never "SystemD."

PID 1

systemd holds process ID 1 — the single root of the entire process tree. Find it with `ps -p 1`.

How a Linux System Boots

Power-on hands control through a short chain until systemd takes over and reaches a target state:



systemd is where the boot process becomes something you manage — everything after it is a unit you can inspect and control.

systemd vs. SysVinit

systemd replaced the older SysVinit approach and does the same job far more efficiently:

Aspect	SysVinit (older)	systemd (modern)
Startup	Runs services one after another	Starts independent services in parallel
Configuration	Collections of init shell scripts	Declarative unit files
Control	Separate, basic scripts	One unified command: systemctl
Result	Slower, less predictable boots	Faster, consistent, manageable

Units and Unit Files

systemd organizes the system into units. A unit is a resource it manages, described by a plain-text unit file whose extension names its type:

Unit type	What it manages
<code>.service</code>	A program systemd starts and supervises — your focus this week
<code>.target</code>	A grouping of units that defines a system state
<code>.timer</code>	A schedule that triggers another unit (an alternative to cron — see Unit 12)
<code>.mount</code>	A filesystem mount point managed by systemd
<code>.socket</code>	A socket that can start a service on demand when traffic arrives

Where Unit Files Live

Two locations, and the distinction matters:

`/usr/lib/systemd/system`

- Vendor defaults installed by packages.
- Read these to learn how a service is configured.
- Do not edit — a package update overwrites them.

`/etc/systemd/system`

- Your customizations live here.
- Files here override the vendor defaults.
- Survives package updates — edit here.

After you add or edit a unit file:

Run **`systemctl daemon-reload`** so systemd re-reads its configuration. Forgetting this is the most common reason a freshly edited service "doesn't work" — systemd is simply still running the old version.

Inside a Service Unit File

A unit file is made of [sections] holding directive=value lines. A service typically has three:

```
[Unit]
Description=Nightly report
After=network-online.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/report.sh

[Install]
WantedBy=multi-user.target
```

[Unit]

Describes the unit and its relationships — a readable Description plus ordering and dependencies.

[Service]

Defines how the program runs: what to execute, how, and as whom. Only service units have this.

[Install]

Controls what enabling does — chiefly which target the unit attaches to so it starts at boot.

Key Service Directives

The directives you'll meet most often inside a service unit:

Directive	What it does
ExecStart= / ExecStop=	The command that starts the service (and how to stop it cleanly).
After= / Before=	Ordering — e.g. After=network-online.target waits for the network.
Requires= / Wants=	Dependencies — Requires= is hard (fails together); Wants= is soft.
Type=	How startup is interpreted — simple keeps running; oneshot runs once and exits.
WantedBy=	In [Install], the target this unit attaches to when enabled.

A typo in a path, a missing dependency, or backward ordering is a leading cause of a service that won't start.

Services and Daemons

The words are used interchangeably, but there's a real distinction worth keeping straight:

Daemon

- A background program, no direct user interaction.
- Runs quietly to do the system's work.
- Name traditionally ends in "d" — `sshd`, `httpd`.

Service

- The capability that program provides.
- The reason a server exists — email, login, web.
- Usually implemented by a daemon.

In practice:

Secure Shell is the service; **`sshd`** is the daemon that provides it. Managing "the SSH service" with `systemd` means managing the unit that supervises the `sshd` daemon.

LESSON 2

Managing Services with `systemctl`



The systemctl Command

One command does nearly all service management, following a `systemctl {subcommand} {unit}` pattern:

Command	What it does
<code>status</code>	Shows running state, recent log lines, and enabled state — your first stop
<code>start / stop</code>	Turns the service on or off now (not persistent across reboot)
<code>restart / reload</code>	Restarts; reload re-reads config without dropping connections
<code>enable / disable</code>	Sets or clears start-at-boot (persistent; does not change current state)
<code>enable --now</code>	Enables for boot and starts immediately, in one step
<code>mask / unmask</code>	Blocks a unit from starting at all (links to /dev/null) — and reverses it
<code>daemon-reload</code>	Re-reads unit files after you add or edit one

Don't confuse `systemctl` (services) with `sysctl` (kernel parameters) — different tools.

Enabling Is Not Starting

The single most-confused distinction in the unit — they are independent settings:

Start

- Turns the service on right now.
- Affects only the current session.
- Reboot, and it reverts to its default.

Enable

- Sets the service to come up at every boot.
- Persistent across reboots.
- Does not turn it on right now.

Want it running now and after every reboot?

Do both at once: `systemctl enable --now sshd`. This is the standard move when you install a service you intend to keep.

Masking Is Not Disabling

A similar trap on the "off" side — and it matters for security:

Disable

- Removes the start-at-boot hook.
- Another service's dependency can still start it.
- A soft "off."

Mask

- Links the unit to `/dev/null`.
- Cannot be started by anyone or anything.
- The strong "off" — until you unmask it.

When it truly must stay down:

Reach for **`systemctl mask`** — for security, or to stop a dependency from reviving a service you turned off.

Live Demo: Manage a Service

LIVE DEMO

Drive the sshd service end to end from the command line.

```
# Where does it stand right now?
systemctl status sshd

# Turn it off for this session, then back on
sudo systemctl stop sshd
sudo systemctl start sshd

# Make it survive a reboot, and confirm
sudo systemctl enable --now sshd
systemctl is-enabled sshd
```

Watch the Active and Loaded lines in status change as you go.

LESSON 3

Boot Targets



What Is a Target?

- **A named system state — a collection of units that together describe how the system is running.**
- Targets are systemd's replacement for the old SysVinit runlevels.
- The most important choice they capture: boot to a plain command line, or to a graphical desktop.

Runlevel mapping

Old runlevel 3 (multi-user text) →
`multi-user.target`
Old runlevel 5 (graphical)
→ `graphical.target`

Common Targets

Target	What it means
<code>multi-user.target</code>	Multi-user command line with networking and services, no GUI — the server default
<code>graphical.target</code>	Everything in multi-user plus a graphical login and desktop — typical workstation
<code>rescue.target</code>	A minimal single-user state for maintenance and recovery
<code>emergency.target</code>	The barest state — a single root shell — for serious repair

On a server, a graphical environment just consumes resources the services should be using — multi-user.target is the right default.

Setting the Default Target

A special unit, `default.target`, points at whichever target the system boots into. You manage it all with `systemctl`:

Command	What it does
<code>systemctl get-default</code>	Shows which target the system boots to
<code>systemctl set-default multi-user.target</code>	Changes the boot default (takes effect next reboot)
<code>systemctl isolate graphical.target</code>	Switches the running system to a target now, without rebooting

get-default reads the setting; set-default changes the next boot; isolate changes the system right now.

LESSON 4

Logs and Troubleshooting



Reading the Journal with journalctl

- **systemd captures every unit's output and status in a central log called the journal.**
- It's maintained by a component named `journald` and read with `journalctl`.
- It's the natural companion to `systemctl status`: `status` says a service is unhappy, the journal says why.

status + journal

`status` gives the headline;
`journalctl -u <unit>` gives the full story. Read them together.

Useful journalctl Options

Command	What it shows
<code>journalctl -u sshd</code>	Only the entries for one unit — the option you'll use most
<code>journalctl -b</code>	Only messages from the current boot
<code>journalctl -f</code>	Follows new entries live, like <code>tail -f</code> (Ctrl+C to stop)
<code>journalctl -p err</code>	Filters to a priority level and worse — here, errors
<code>journalctl --since "10 min ago"</code>	Limits to a time window; pairs with <code>--until</code>
<code>journalctl -xe</code>	Jumps to the end with extra hints — fast view of what just broke

Options combine: `journalctl -u sshd -b -p err` shows only SSH's errors from this boot.

This is your introduction to the journal — Unit 11 goes deeper into log analysis.

Troubleshooting a Service That Won't Start

Resist the urge to guess. A short, repeatable routine resolves most problems:

- 1 Check the status** `systemctl status {svc}` — running, failed, or never tried, plus the last log lines.
- 2 Read the journal** `journalctl -u {svc} -b` — this boot's full log; errors usually announce themselves.
- 3 If you edited a unit, reload** `systemctl daemon-reload` so systemd picks up the change.
- 4 Restart and re-check** `systemctl restart {svc}` — the common fix after a config change; confirm with status.
- 5 Confirm it's enabled** If it vanishes after reboot, it was started but never enabled.

Finding Slow Services

When a server boots slowly, one service is usually holding everything up. systemd can name it:

```
$ systemd-analyze blame
18.4s  NetworkManager-wait-online
 6.1s  dnf-makecache
 2.3s  sshd
 0.9s  firewalld
...
```

- Lists every unit by start time, longest first.
- Investigate the unit at the top of the list.
- The cause is often a misconfiguration in that unit's file.

Live Demo: Break It, Then Fix It

LIVE DEMO

Introduce a fault, watch it fail, and diagnose it from the journal.

```
# Point ExecStart at a path that doesn't exist, then reload
sudo systemctl edit --full demo.service
sudo systemctl daemon-reload

# Watch it fail, then find out why
sudo systemctl restart demo.service      # fails
systemctl status demo.service            # Active: failed
journalctl -u demo.service -b            # the real reason

# Fix the path, reload, restart — back to active
```

LESSON 5

Writing Your Own Service



Creating a Simple Custom Service

Bring your own program under systemd in four steps:

- 1 Write the unit file** Create `/etc/systemd/system/hello.service` with [Unit], [Service], [Install].
- 2 Reload systemd** `systemctl daemon-reload` so it reads the new file.
- 3 Start and enable it** `systemctl enable --now hello.service` — on now, and at every boot.
- 4 Verify** `systemctl status hello.service` and `journalctl -u hello.service`.

That's the exact pattern real administrators use to run custom tools as first-class services — and what you'll do in the lab.

Security Focus: Every Running Service Is a Door



Run only what the server actually needs — the most effective security control there is.

- Disable services you don't need — fewer running services means fewer things to patch, watch, and defend.
- Mask, don't just disable, when it matters — a disabled service can still be pulled in by a dependency.
- Know what's listening — pair service management with `ss -tulpn` to see which services hold network ports open.
- Read the journal as a detection tool — repeated failed starts or an unfamiliar unit can be your first warning.

Why it matters: the smallest set of running services is the smallest attack surface. You'll formalize this with firewalls and SELinux in Unit 11 — the habit starts here.

Rocky and Ubuntu: One systemd

systemd is the happy exception in a dual-ecosystem course — it works the same on both:

- systemctl and journalctl, the unit-file format, section and directive names, and the standard targets are identical.
- A skill you build on one transfers exactly to the other.

Dual-Distro Note

The SSH unit is named differently: **sshd.service** on Rocky (Red Hat style) but **ssh.service** on Ubuntu (Debian style). The other real difference is which units ship enabled by default — Rocky enables `firewalld`, Ubuntu does not. The commands are the same; confirm a unit's exact name with `systemctl list-units --type=service`.

This Week's Lab

NETLAB+ Lab: Services and Systemd — performed on both Rocky and Ubuntu.

- Manage services with systemctl — start, stop, enable, and disable system daemons.
- Inspect and edit a unit file, then reload systemd to apply the change.
- Write and activate a simple custom service of your own.
- Change the system's default boot target.
- Read a service's logs with journalctl.

Starting point & submission

Start from the **WK09_SYSTEMD** snapshot. You're graded on the observable end state — capture each required command's output before your reservation ends.

Key Takeaways

- **systemd is the init system — PID 1 — and the manager you use for everything afterward.**
- Units are described by unit files; your edits go in `/etc/systemd/system`, then `daemon-reload`.
- `systemctl` drives it all. Enabling $\neq$ starting; masking $\neq$ disabling.
- Targets are named system states — `multi-user.target` for servers, `graphical.target` for desktops.
- `journalctl -u <unit>` is the window into why a service behaves the way it does.
- Run only what's needed — every service is a door, and the journal helps you watch them.

Questions?

Unit 9 · Services and Systemd

